WEEK 4 · DAY 2 · RULE BOOK

Meet scikit-learn + Isolation Forest

Follow it step by step. Every common error and its fix is here.

Your goal today: train your first machine-learning model, see what it finds, and check **why** it found it. **15 green ✅ ticks** to collect (14 + the 🌶️ pepper task). Time: about 60 minutes. Keep this page open next to Colab.

Before you start (2 minutes)

- ☐ Open `W4D2-exercise` in Colab → **File** → **Save a copy in Drive**.
- ☐ Run **STEP 0**. Wait for ✅ `Ready!`
- ☐ Run the ▶️ loading cell. You should see `hours: 8760` and the 5 events of the answer key.
- ☐ Run every ▶️ cell in order. Each ✍️ cell depends on the cells above it.

The scikit-learn pattern (every model, every time)

Step	Code	In plain words
1. Build X	`X = bus[CHANNELS]`	A table of numbers: **rows = hours**, **columns = channels**
2. Make a model	`IsolationForest(n_estimators=200, contamination=0.005, random_state=42)`	200 random trees · allowed to flag 0.5% · same result every run
3. Fit	`model.fit(X[train])`	Learn what "normal" looks like from January–August. **No labels.**
4. Ask	`model.predict(X)`	**-1 = odd**, 1 = looks normal
5. Score	`model.decision_function(X)`	A number per hour: **lower = more odd**

Three states, three normals

State	When	Normal looks like	Judged today?
🌘 Eclipse	21:00 UTC, in the eclipse seasons	array ≈ 0 A, bus sags, battery discharging	❌
🔋 Recharge	the 3 hours after an eclipse	array ≈ 29 A, battery warm, SoC climbing	❌

State	When	Normal looks like	Judged today?
🌞 Steady sun	everything else	array ≈ 28 A, SoC 100%, bus ≈ 50.4 V	✅

```
steady = bus["eclipse_flag"].rolling(4).max() == 0     # no eclipse in this hour or the 3 before
```

The exercise, step by step

Task	What you do	The tool you need
A1	Build X	`bus[CHANNELS]`
A2	The training months	`TRAIN_END = "2025-09-01"`
A3	Make and fit the forest	`CONTAMINATION = 0.005`
A4	How many flags should you expect?	`CONTAMINATION * len(X)`
A5	Which `predict()` value is a flag?	`-1`
B1	What hour are the flags?	read `value_counts()`
B2	Score the forest	`score_events(flags_all, labels)`
B3	Real catch or lucky catch?	`True` or `False` (no quotes)
C1	How many hours in the steady-sun window?	this hour + 3 before
C2	Forest on steady sun only	`train & steady`
C3	The Week 3 chart on the same hours	`K = 3`
C4	Who caught EV03?	`"forest"` or `"chart"`
D1	k-means finds the modes	`StandardScaler()`
D2	Explain a flag	the channel with the biggest z (ignore the sign)
🌶️ **Bonus**	Turn the contamination dial down	try `0.002`

The two helpers (you'll see them in the notebook)

```
score_events(flags, labels)      # the Week 4 scorer: caught, false_alarms, recall, precision
event_table(flags, labels)       # one row per event: caught? hours_late? hours_flagged of hours_long?
```

`flags` is a True/False Series with the same time index as `bus`. **Always look at `event_table`**: a long event can be "caught" by a flag that has nothing to do with it (a *lucky catch*).

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again
`NameError: name 'IsolationForest' …`	You skipped the loading cell	Run the ▶️ cell that loads `bus` and imports scikit-learn
`NameError: name 'score_events' …`	You skipped the scorer cell	Run the ▶️ scorer cell in Part B
`flagged hours: 8702` (thousands)	You flagged the **normal** hours	Odd is `-1`, not `1`
`InvalidParameterError: … contamination`	contamination isn't a small fraction	Use `0.005` (that's 0.5%), not `5` or `0.5`
Your forest numbers differ by 1–2 from a neighbour's	Different `random_state` or scikit-learn version	Use `random_state=42`. Small differences are normal; the checks allow them
`TypeError: '<' not supported …`	`TRAIN_END` is a number	Write the date as text, in quotes: `"2025-09-01"`
`KeyError: 'eclipse'` in D1	You passed something other than the crosstab	`check("D1", table)`
D1 ❌ even though it ran	k-means without the scaler	`SCALER = StandardScaler()` (with the brackets)
`ValueError: Invalid frequency: H`	Old code with capital `"H"`	Use lowercase `"h"`
Everything is weird	–	**Runtime → Restart session and run all**

Rules of thumb you'll keep forever

1. **Sanity-check the count** before you look at the results: contamination × rows ≈ flags. Thousands = a bug.
2. **Rare is not broken.** A forest finds what is unusual; eclipse is unusual and perfectly normal.
3. **Give the model context** (steady sun) instead of hoping it works it out.
4. **Check *why* it flagged:** which hours, what time of day, which channel. A lucky catch is not a catch.
5. **Explain every flag** with per-channel z-scores. The biggest one is your sentence for the duty engineer.
6. **Scale before distance methods** (k-means). The forest doesn't need it, but the pipeline habit never hurts.

7. **Set** `random_state` so your result is the same tomorrow, and the same as your neighbour's.
8. **The old chart still wins some fights.** A single-channel spike is the chart's home ground.

🏁 **Finished?** Run `scoreboard()`. Then try the 🌶️ dial, and ask yourself: the forest saw only 1 hour of EV04. What column would it need to see "hot for its workload"?